

Frontend Web — Angular 21

Aplicación web con **Angular 21**, renderizado del lado del servidor (SSR), detección de cambios **zoneless**, comunicación GraphQL vía **Apollo Angular**, UI con **PrimeNG 21** + **Tailwind CSS 3** y soporte **PWA** completo.

Comandos principales

```
npm start                # Dev server → http://localhost:4200
npm run build             # Build de producción
npm run build:cloudflare  # Build para Cloudflare Pages
npm test                 # Tests con Vitest
npm run test:cov          # Tests con cobertura
npm run lint              # ESLint --fix
npm run format            # Prettier --write
npm run compodoc:serve    # Documentación con Compodoc

ng generate component features/mi-feature/mi-componente
```

Stack

Capa	Tecnología	Detalle
Framework	Angular 21	Zoneless, SSR con <code>@angular/ssr</code>
API	Apollo Angular	GraphQL, queries tipadas
Auth / Storage	Supabase JS	Sesión, almacenamiento de imágenes
UI Components	PrimeNG 21	Preset Aura, paleta naranja customizada
Estilos	Tailwind CSS 3	Utility-first, dark mode vía <code>data-theme</code>
Estado	Signal Stores	<code>signal()</code> , <code>computed()</code> , zoneless
Testing	Vitest	Unitario + cobertura
Calidad	ESLint + Prettier + Husky	lint-staged en pre-commit

Aliases de path

Alias	Ruta real
<code>@core/*</code>	<code>src/app/core/*</code>
<code>@shared/*</code>	<code>src/app/shared/*</code>
<code>@features/*</code>	<code>src/app/features/*</code>
<code>@graphql/*</code>	<code>src/app/graphql/*</code>
<code>@env/*</code>	<code>src/environments/*</code>
<code>@testing/*</code>	<code>src/testing/*</code>

Arquitectura de capas

```
flowchart TD
    subgraph UI["Capa UI (features/)"]
        Pages["Páginas\n(lazy-loaded)"]
        Shared["Componentes compartidos\n(post-card, topbar, avatar...)"]
    end

    subgraph State["Capa de Estado (store/)"]
        Stores["Signal Stores\n(FeedStore, AuthStore, ChatStore...)"]
    end

    subgraph Domain["Capa de Dominio (services/)"]
        Services["Servicios\n(PostService, UserService...)"]
    end

    subgraph Data["Capa de Datos (repositories/)"]
        Repos["Interfaces IRepository\n(IPostRepository, IUserRepository...)"]
        GQL["Impl. GraphQL\n(PostGraphQLRepository)"]
        SBRepo["Impl. Supabase\n(UserSupabaseRepository)"]
    end

    subgraph External["Externo"]
        API["API GraphQL\n(Cloudflare Worker)"]
        SB["Supabase\n(Auth / Storage)"]
    end

    Pages --> Stores
    Stores --> Services
    Services --> Repos
    Repos --> GQL --> API
    Repos --> SBRepo --> SB
```

Estructura de carpetas

```
src/app/
├─ core/
│   ├─ guards/          authGuard, guestGuard (canActivate + canMatch)
│   ├─ interceptors/    auth (añade JWT), metrics
│   ├─ models/          interfaces de dominio (Post, User, Recipe...)
│   ├─ repositories/    interfaces + implementaciones concretas
│   ├─ services/        lógica de negocio (TranslationService, ContentTranslationS
│   ├─ store/           Signal Stores por dominio
│   ├─ tokens/          InjectionTokens (ENVIRONMENT, POST_REPOSITORY...)
│   └─ utils/           funciones puras
├─ features/            páginas lazy-loaded por dominio
├─ graphql/             queries y mutations Apollo tipadas
└─ shared/
    ├─ components/      app-shell, topbar, post-card, avatar, comment-section...
    ├─ directives/      click-outside, img-fallback
    └─ pipes/           translate, time-ago, truncate, message-markdown
```

Rutas y páginas

```
flowchart LR
    Auth["/auth\nLogin / Registro"]

    subgraph App ["App (autenticado)"]
        Feed["/\nFeed"]
        Explore["/explore\nExplorar"]
        AI["/ai\nGenerador IA"]
        Saved["/saved\nGuardados"]
        Chat["/chat\nMensajes"]
        Profile["/profile\nPerfil propio"]
        PubProfile["/profile/:username\nPerfil público"]
        Settings["/settings\nAjustes"]
        Post["/post/:id\nDetalle post"]
        Cook["/cook/:id\nModo cocina"]
        Shopping["/shopping\nLista compra"]
        Places["/places\nLugares"]
        PlaceDetail["/places/:id\nDetalle lugar"]
    end

    Auth --> Feed
```

Ruta	Página	Acceso
<code>/auth</code>	Login y registro	Público
<code>/</code>	Feed principal	Autenticado
<code>/explore</code>	Exploración de contenido	Autenticado
<code>/ai</code>	Generador de recetas con IA	Autenticado
<code>/saved</code>	Publicaciones guardadas	Autenticado
<code>/chat</code>	Mensajes directos	Autenticado
<code>/profile</code>	Perfil propio	Autenticado
<code>/profile/:username</code>	Perfil público	Autenticado
<code>/settings</code>	Ajustes de cuenta	Autenticado
<code>/post/:id</code>	Detalle de publicación	Autenticado
<code>/cook/:id</code>	Modo cocina (paso a paso)	Autenticado
<code>/shopping</code>	Lista de la compra	Autenticado
<code>/places</code>	Lugares gastronómicos	Autenticado
<code>/places/:id</code>	Detalle de lugar	Autenticado

Patrón Repository

Cada dominio tiene una interfaz, una o varias implementaciones y un token de inyección. Los servicios nunca dependen de la implementación concreta.

```

flowchart LR
    Service["PostService\n(inyecta por token)"]
    Token["POST_REPOSITORY\n(InjectionToken)"]
    Interface["IPostRepository\n(interfaz)"]
    GQL["PostGraphQLRepository"]
    SB["PostSupabaseRepository"]
    Config["app.config.ts\nprovide: POST_REPOSITORY\nuseClass: PostGraphQLRepository"]

    Service -->|inject| Token
    Token --> Interface
    Interface --> GQL
    Interface --> SB
    Config -.→|binding|. Token

```

```

// Servicio - solo conoce la interfaz
constructor(@Inject(POST_REPOSITORY) private repo: IPostRepository) {}

// app.config.ts - decide la implementación
{ provide: POST_REPOSITORY, useClass: PostGraphQLRepository }

```

Cambiar el backend (GraphQL ↔ Supabase) es cambiar una línea en `app.config.ts`.

Signal Stores (gestión de estado)

```
@Injectable({ providedIn: 'root' })
export class FeedStore {
  // Estado privado
  private _posts = signal<Post[]>([]);
  private _loading = signal(false);

  // Estado público (readonly)
  readonly posts = this._posts.asReadonly();
  readonly loading = this._loading.asReadonly();

  // Estado derivado
  readonly isEmpty = computed(() => this._posts().length === 0);

  async loadFeed() {
    this._loading.set(true);
    const data = await this.postService.getFeed();
    this._posts.set(data);
    this._loading.set(false);
  }
}
```


Internacionalización (i18n) — 14 idiomas

```
flowchart LR
    User["Usuario\ncambia idioma"]
    TS["TranslationService\ncurrentLang Signal"]
    LS["localStorage\nsave_real_lang"]
    Pipe["TranslatePipe\n{{ 'key' | translate }}"]
    Components["Componentes\nre-renderizan"]

    User --> TS
    TS --> LS
    TS --> Pipe
    Pipe --> Components
```

```
User → TS
TS → LS
TS → Pipe
Pipe → Components
```

Idiomas soportados

Código	Idioma	Regiones
es	Español	España, México
en	English	EEUU, UK, Canadá, Australia, Singapur
fr	Français	Francia, Suiza
de	Deutsch	Alemania, Suiza
it	Italiano	Italia
pt-BR	Português	Brasil
nl	Nederlands	Países Bajos
sv	Svenska	Suecia
no	Norsk	Noruega
ja	日本語	Japón
ko	한국어	Corea del Sur
ar	العربية	Emiratos Árabes
hi	हिन्दी	India
id	Bahasa Indonesia	Indonesia

Uso en código

```
// En componente
this.translationService.currentLang()      // → 'es' | 'en' | 'ja' | ...
this.translationService.setLanguage('ja')  // persiste en localStorage
this.translationService.translate('auth.login_btn') // → 'ログイン'
```

```
←!— En templates →  
{{ 'shell.explore' | translate }}  
{{ 'auth.register.create_btn' | translate }}
```

Añadir nuevas claves

Añadir la clave en los **14 bloques** de `translation.service.ts`:

```
// En cada uno de los 14 bloques (es, en, fr, de, it, pt-BR, nl, sv, no, ja, ko, etc.)  
'mi_seccion.mi_clave': 'Texto en el idioma correspondiente',
```

Traducción de contenido dinámico

Las descripciones de posts y los mensajes de chat se traducen en tiempo real.

```

sequenceDiagram
    actor U as Usuario
    participant CT as ContentTranslationService
    participant Cache as Caché (Map)
    participant API as MyMemory API

    U->>CT: translate("Ciao mondo")
    CT->>Cache: ¿Existe "en::Ciao mondo"?
    alt En caché
        Cache-->>CT: "Hello world"
    else Sin caché
        CT->>API: GET /get?q=Ciao mondo&langpair=auto|en
        API-->>CT: { responseData: { translatedText: "Hello world" } }
        CT->>Cache: Guarda "en::Ciao mondo" → "Hello world"
    end
    CT-->>U: "Hello world"

```

MyMemory API: gratuita, sin API key, límite 5.000 palabras/día por IP. Soporta RTL para árabe (`dir="rtl"`).

SSR — Guarda de APIs de navegador

```

// ✅ Correcto – protege código browser-only
constructor(
  @Inject(PLATFORM_ID) private platformId: object
) {}

someMethod() {
  if (isPlatformBrowser(this.platformId)) {
    localStorage.setItem('key', 'value'); // solo en browser
  }
}

```

Variables de entorno

`src/environments/environment.ts`:

```
export const environment = {  
  production: false,  
  supabaseUrl: 'https://<project-ref>.supabase.co',  
  supabaseKey: '<supabase-anon-key>',  
  apiUrl: 'http://localhost:8787',    // ← local  
  // apiUrl: 'https://develop.app.savorealo.com/api', // ← producción  
}
```